Cloud Computing Mini Project

Name: Lokesh Yadav
Roll number: 230101039
Repository: lokesh1-cell/cloud-assignmnet
Course: CS3081 — **Measured:** 8 November 2026

A CPU-only command parser distils Phi-3.5 into Pythia-70M. The pipeline is command pool -> parallel teacher workers -> strict cleaner -> student training -> S3 model storage -> FastAPI -> Kubernetes Service/Ingress. All current numbers below come from this repository's new executions; supplied earlier logs are preserved separately in `results/historical/`. No validation labels were used for training. Raw model text is scored without JSON repair.

Task 1: Object storage and parallel labelling

`cluster/k3d.yaml` creates k3d; SeaweedFS runs in `data` with buckets `datasets` and `models`. The command pool is uploaded before labelling. `prompts/teacher_prompt.txt` requests exactly the four keys, the eight allowed intents, literal minimal slot spans and null for absent slots. Ollama 0.40.15 serves the required `phi3.5` teacher with temperature 0 and seed 42. Teacher replies are retained exactly, including whitespace.

Workers in `pipeline` select zero-based input lines using line_number modulo SHARD_COUNT = SHARD_INDEX. To finish CPU labelling using free runners, two runner groups each ran two concurrent workers, globally SHARD_COUNT=4 with indexes 0/1 and 2/3. Each wrote 750 rows to its own S3 shard. Existing genuine checkpoints were resumed; transient transport failures were retried without editing successful replies. The four shards were collected into shared object storage before cleaning. Actual coverage is 3000 rows, 3000 unique command IDs, zero gaps/repeats and zero empty replies. The evidence includes worker logs, shard coverage and object listings.

Cleaning category (first applicable reason)	**Records**
Input	3000
(a) Invalid JSON	0
(b) Incorrect keys	1
(c) Unsupported or non-string intent	4
(d) Non-null slot not an exact nonempty command span	98
Duplicate commands, retain smallest ID	49
Kept, sorted by ID	2848

3000 = 0 + 1 + 4 + 98 + 49 + 2848. Local re-cleaning reproduced the dataset byte for byte. SHA-256 is `5cda236a5762e5b0d56b0d502be90410e3b321205f5007b18c7154d453823f40`. Evidence: `results/fresh/local_dataset_verification.json`, training/data artifacts, and teacher generation run. The parent aggregate stage was intentionally cancelled after both label jobs and teacher evaluation succeeded, so corrected cleaning/training could consume those same artifacts. Static sharding can leave stragglers; a resumable queue would improve utilisation.

Task 2: Sequence-level student distillation

`make train` reads `training/config.yaml`: EleutherAI/pythia-70m, seed 42, learning rate 0.00005, batch size 8, three epochs, maximum sequence length 128, and only the first 1500 cleaned examples. Training strings are `Command: <text>` followed by `JSON: <teacher reply>`. CPU PyTorch 2.4.1 and Transformers 4.44.2 are pinned; the exact environment is in `training/requirements.lock.txt`. The 50-example smoke run completed before full training. Version v2 uses the same data/config and seed 7. The 100 labelled validation commands are used only for evaluation.

Model (100 validation commands)	JSON valid	Intent	Date	Time	Place	Full frame
Teacher phi3.5	100%	85%	80%	88%	86%	49%
Untrained Pythia-70M	0%	0%	0%	0%	0%	0%
Student v1, seed 42	100%	79%	76%	90%	82%	41%
Student v2, seed 7	99%	83%	76%	87%	82%	42%

`results/pred_*.jsonl` and score JSON files retain all 100 raw outputs per model, with no missing IDs. Re-running the provided scorer exactly reproduces these tables; release-based CI independently reproduced both student scores. V1 epoch losses are 0.241773, 0.095099, 0.076223; v2 losses are 0.247422, 0.096295, 0.074788. V1's 41% full-frame result exceeds half of the teacher's 49% on this validation set; hidden-test performance is unknown. The student learns JSON structure but still confuses intents and slots. V2 produces an invalid repeated intent on V091; it is retained and counted as an error.

Both distinct models, tokenizers, version metadata, archive checksums and individual file manifests are published in release model-v1. Each version's seven manifest entries and archive hash were verified locally and by streaming the S3 objects; lineage metadata records 1500 examples and the training-data checksum above. `models/v1/model.sha256` identifies the release archive. No weights are committed to Git or embedded in the image. Evidence: successful training run, `results/fresh/local_model_verification.json`, and `results/fresh/live/storage_verification.json`.

Sequence-level distillation trains on the teacher's generated sequences as ordinary targets. Logit-based distillation instead matches soft token probability distributions, often using a temperature-scaled divergence. This project's saved Ollama generation responses contain text, not full teacher token distributions; Phi and Pythia also have different token vocabularies, so directly matching their logits is undefined without alignment and a different teacher interface. This implementation therefore cannot perform logit matching. A hard target discards uncertainty and transfers teacher mistakes. A 70M causal model is small enough for CPU deployment but has limited capacity and copying accuracy; better prompt design and error analysis could improve labels without substituting hand-written answers.

Task 3: Container inference service

FastAPI listens on port 8000. `/healthz` returns 200 while alive; `/readyz` returns 503 until model loading completes, then 200. `/parse` validates the raw decoded model output against `data/schema.json`; success contains only intent/date/time/place, with `X-Model-Version` and Pod-name `X-Served-By` headers. Malformed output yields HTTP 500 and exactly {`"error":"invalid_model_output"`}. The live fault-injection request confirmed this body and status. Unready parse requests return 503.

MODEL_URI, MODEL_VERSION, S3_ENDPOINT and MAX_NEW_TOKENS control loading/generation; FAULT_INJECT tests validation. Credentials come from a Kubernetes Secret, not source files. The loader paginates S3 listings, rejects unsafe object paths, downloads the model/tokenizer on startup, and logs each download. CPU inference uses one Torch thread. Dockerfiles use an immutable Python base digest, CPU-only Torch, and fully frozen direct/transitive dependencies.

The fresh live build measured **1,267,334,183 bytes**, below 2 GB, with no model weights in `/app`. A separate container was limited to **1 CPU and 1 GiB**, with swap also capped at 1 GiB. All **50 sequential requests succeeded**, OOMKilled=false, exit code 0. The cgroup memory peak was **793,964,544 bytes (757.18 MiB)**; this is distinct from sampled Kubernetes working-set memory. Evidence: `results/fresh/live/limited_container_*`, image history/files/size, and fault result in `live_summary.json`. A sampled metric can miss short peaks; cgroup peak accounting addresses that for this container test. Production hardening would add authentication and admission limits.

Task 4: Kubernetes scaling and reliability

The `app` namespace contains a Deployment, Service and Traefik Ingress. Two replicas request 250m CPU/512 MiB each and are limited to 1 CPU/1 GiB. Liveness uses `/healthz`, readiness uses `/readyz`; readiness endpoint observations show zero routable endpoints before the replacement models become ready. Rolling updates use maxUnavailable=0 and maxSurge=1; a five-second preStop drain and 45-second termination grace reduce in-flight failures. HPA minimum=1, maximum=2, target CPU=70%, with a 30-second downscale stabilisation window.

Actual experiment	Measurement	Required outcome
Three replacement cohorts, creation to Ready	9 s, 8 s, 9 s; median 9 s	Logs explicitly show S3 downloads
HPA load/idle cycle	1 -> 2 -> 1; idle wait 41.77 s	Both scaling directions observed
Roll v1 -> v2 -> undo under load	CSV: 548 requests, 1 failure = 0.1825%	At most 0.5%; restored header v1
Delete one Pod under load	CSV: 354 requests, 0 failures; restored in 7.42 s	At most 2%; two Ready replicas restored
Two-replica load balancing	180 and 176 successful final responses	Shares 50.56% and 49.44%, each above 30%

Direct requests after update and rollback returned model headers v2 and v1 respectively. The rollout's one real failure is included; Locust correctly exited 1 because any failure triggers that exit code. All thresholds above are calculated from actual evidence rather than the workflow conclusion alone. Logs, CSVs, endpoint/HPA snapshots and teardown output are in `results/fresh/live/`; live run completed successfully.

The k3d serving cluster ran on one free runner VM (four logical CPUs, not multiple physical fault domains). Container/Pod replacement demonstrates local process recovery, not machine-level availability. Three cold starts are a small sample and exclude image pull/build time because timing starts at Pod creation. Multi-host deployment and longer perturbation tests would strengthen reliability evidence.

Task 5: Subnetting and stateless access control

Largest-first contiguous VLSM uses usable hosts = 2^(32-prefix)-2. The next smaller sizes /23, /25, /26, /28 offer only 510, 126, 62, 14 hosts respectively, so the selected prefixes are minimal. Detailed working is in `network/subnets.md`.

Subnet / required hosts	Network	Usable range	Broadcast	Usable hosts
app / 1000	10.20.0.0/22	10.20.0.1-10.20.3.254	10.20.3.255	1022
public / 250	10.20.4.0/24	10.20.4.1-10.20.4.254	10.20.4.255	254
data / 100	10.20.5.0/25	10.20.5.1-10.20.5.126	10.20.5.127	126
management / 20	10.20.5.128/27	10.20.5.129-10.20.5.158	10.20.5.159	30

`network/nacl.json` denies all inbound traffic from 198.51.100.0/24 at rule 100, before web-port allows. It allows inbound TCP 80/443 from anywhere, SSH only from management, and ephemeral reply ports 1024-65535 only from app. Outbound permits TCP 8000 to app and ephemeral client reply ports to anywhere; unmatched traffic is denied. A stateless NACL remembers no connection, so each reply direction needs an explicit rule. Lowest-number-first matching makes the abusive-range deny override the broader web allows. The provided simulator passes **6/6** sample packets. The 20 hidden packets are unavailable; these are simulated design rules, not a claim of enforcing VLSM inside k3d. Network-policy enforcement would be a separate improvement.

Task 6: Reproducibility, tests and CI/CD

From a fresh clone: copy `.env.example` to `.env`, set a local development secret, then `make up`. That script creates the cluster, builds/imports images, deploys SeaweedFS, downloads release artifacts with checksum verification, seeds buckets and deploys the service. `make down` removed the tested cluster; the recorded cluster list confirms teardown. Build, training, evaluation, testing, rollout and shutdown commands are in README and `docs/RUNBOOK.md`.

GitHub Actions runs lint, **30 unit/API regression tests**, actual release-model inference, then the supplied scorer with `--min-validity 95`; a separate Gitleaks job scans Git history. Tests include exact teacher whitespace, malformed intent types, exact slot spans, schema failures, readiness and response headers. Frozen training dependencies were captured from the successful run. Release v1 CI and release v2 CI both passed, including the 95% gate and clean secret scan. The deliberately bad untrained-model run failed at that gate with 0% validity after lint/tests succeeded; a missing artifact is not used as a substitute demonstration.

`AI_USE.md` records Codex assistance with auditing, fixes, orchestration and report writing, and how outputs were checked. Existing application logic was retained except for documented correctness/reproducibility fixes: unedited text, type-safe validation, bounded transport retry, secure model download paths, inference thread limits and graceful termination. Random seeds and locks improve reproducibility but cannot guarantee bitwise-identical

training across CPU/library changes. Hidden tests and the student's personal handwritten reflection remain outside automated verification.

Task 7: Performance, cost, SLO and review

The standard two-minute, four-user Locust run used the provided `tools/locustfile.py` through Ingress with exactly two Ready replicas. The CSV snapshot reports **353 requests, zero failures, p50 1200 ms, p95 2500 ms, throughput 2.942516 requests/s**. Thus error rate 0% <=1% and p95 <=5000 ms. Shutdown drained three in-flight requests: the final console/header snapshot instead reports **356 requests, zero failures, p50 1400 ms, p95 2500 ms** and replica totals 180+176. The original files are unchanged. CSV and final console are different snapshots; their denominators are not combined. Rollout/recovery CSV snapshots likewise precede their final header counts.

During the standard run, `kubectl top` samples for Pod suffix 5cglq ranged **369-1009m CPU, 335-442 MiB**; jvg2g ranged **581-779m CPU, 378-409 MiB**. These are sampled metrics, not continuous peaks; short CPU accounting intervals may slightly exceed the nominal 1000m reading. Cold-start median is 9 s. Resource ranges and calculations are reproducible from `results/performance_analysis.json` and the underlying raw samples.

Estimated cost (hypothetical rate card): each replica/hour = 1 vCPU x 3.50 + 1 GiB x 0.45 = 3.95 INR. Two replicas/hour = 7.90 INR. CSV requests/hour = 2.9425162385 x 3600 = 10593.05846. Cost per 1000 = 7.90 / 10593.05846 x 1000 = **0.74577 INR**. This bills service limits; it is not a real provider quote and excludes teacher training, storage, ingress and host overhead.

SLO: at least 99% success over one two-minute standard run. For the explicitly chosen CSV snapshot N=353, error budget = 0.01 x 353 = 3.53 requests, so at most three whole failures are allowed. Zero were used (0% of budget). The final drained snapshot separately has N=356 and budget 3.56; it also has zero failures. Neither demonstrates a monthly availability SLO.

Concern	Measured evidence	Limitation	Proposed improvement
Data and storage	3000 covered, 2848 kept	103 rejected labels; static shards can stall	Improve prompt and use resumable queue
Model quality	v1 full frame 41%, teacher 49%	Noisy hard labels; hidden quality unknown	Analyse intent/slot errors; tune only training data
Latency / throughput	p95 2500 ms, 2.943 rps	Small CPU-only sequential decoder	Evaluate batching under same limits
Resource utilisation	Container peak 757.18 MiB	Close to 1 GiB; top is sampled	Profile allocations and benchmark quantisation
Reliability / recovery	Recovery 7.42 s; 0% failures	One host; only one deletion experiment	Test independent hosts and repeated failures
Security and cost	Secret findings 0; 0.74577 INR/1000	Local secret, no API auth; partial cost model	Rotate secrets, add auth/TLS, include infrastructure costs

Submission: source, results and report are repository deliverables; weights/tokenizers/checksums are Release assets. The student supplies the genuine handwritten scan `230101039.pdf` (at most three pages). Package it with this report, README, final sources and datasets as `230101039.zip`. The technical commit is tagged `final`; hidden-test scores and handwritten authorship are not asserted by this report.